

```
In [2]: # %pip install nltk
```

```
In [3]: import re
import time
import requests

import nltk
from nltk.lm import MLE
from nltk.lm.preprocessing import padded_everygram_pipeline
```

# LLM pre-training + Data Selection

Brendan Tomoschuk

Download today's lecture, as a notebook or pdf

[tomoschuk.github.io/downloads](https://tomoschuk.github.io/downloads)



## Introduction

- 🇮🇹 Senior Data Scientist, Reddit
- 🎓 Lecturer, UCSD (HDSI and Psychology Department)
- Outside the classroom: my dog & husband, dungeons and dragons 🐉, rock climbing 🧗



## What class you're in:

- This class is a lower division **"Language and Modeling"** course
- Class objectives:
  - introduce students to both human language and language models
  - demystify LLMs
  - code in python
  - set up upper division classes in cognitive science, linguistics and data science
- Prerequisites:
  - intro python /pandas
  - intro statistics
  - Not necessarily linear algebra etc.

## Syllabus

Course description: How do humans learn and use language — and how do large language models do it differently? Through weekly Python labs and discussions grounded in linguistics and cognitive science, students build and evaluate small language models to demystify the technology behind tools like ChatGPT and Claude.

- **Week 1:**
  - What is language? What is a language model?

- Python set up, NLTK package
- **Week 2:**
  - Bag of words, TF-IDF
- **Week 3:**
  - **LLM pre-training** (You are here!)
  - Statistical learning
- **Week 4:**
  - LLM fine-tuning, LLM alignment
  - Supervised vs unsupervised learning
- **Week 5:**
  - Syntax and pragmatics
  - Self-attention
- **Week 6:**
  - Distributional Semantics
  - embeddings
- **Week 7: Model evaluation 1**
  - Construct validity
  - Surprisal & Perplexity
  - Theory of mind, other cognitive tests
  - LLM benchmarks and leaderboards
- **Week 8:**
  - Context windows, working memory
  - Transformers
- **Week 9:**
  - Sociolinguistics
  - Data input and bias
- **Week 10:**
  - Open vs closed inputs
  - Reasoning models
  - Review & prepare for final

## In class today:

- Breakdown LLM pre-training:
  - Corpus
  - Tokenize
  - Train
  - Evaluate
- Train our own language model
- Discuss how data biases models
- Highlight parallels between language models and human language

## Have a question? Ask it!

- I've baked in time for questions and interactions.
- Prefer anonymity? Ask questions at the [google form](#) below and I'll check periodically.
- This form also allows me to ask you questions and chat through your answers!



## Why should you learn about LLMs?

- You want to research cognition?
  - LLMs are helping us build a better understanding of human language processing, vision etc.
- You want to research LLMs?
  - Cognitive science is on the cutting edge of how we build and measure these tools
- You are worried about how LLMs are impacting society?
  - Understanding the fundamentals of LLMs help us understand their limitations, inform regulation etc

## What is a language model?

- You know language models like ChatGPT, Claude etc
- These are a specific type of large language model (LLM), which is a machine learned model that processes natural language (e.g. English)
- Training of these models is typically split into three stages. Today we will focus on **pre-training**

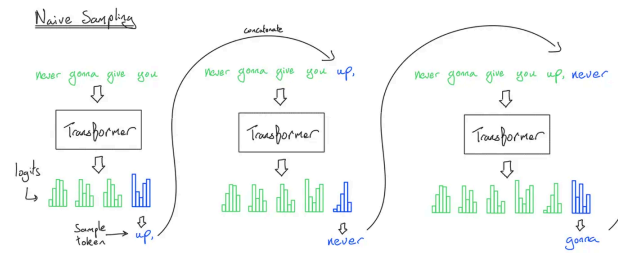
# Pre-training

# Fine-tuning

# Alignment

## What is pre-training?

- LLM learns to **predict** language.
- **Self-supervised**
  - Model creates its own labels from the data
- It uses billions or trillions of words (or tokens), and often millions of dollars in compute costs
- The price of this stage is what causes LLMs to be driven by for-profit entities



Today we are going to train our own language model! An N-gram model which is a precursor to the LLM. We'll and talk about how they're similar and different.

## Predictive language models:

- Corpus — load in some language data
- Tokenize — turn raw data into usable units
- Train — build the model
- Evaluation — examine the output

Corpus

Tokenize

Train

Evaluate

## Loading a corpus

- **corpus** : one or more documents containing language. For this class we'll use a collection of public domain books from [Project Gutenberg](#), in this case the complete works of Jane Austen.

- Let's **Download** plain UTF-8, **strip** the standard Gutenberg header/footer and look at some basic stats about our data before going any further

Corpus

Tokenize

Train

Evaluate

```
In [4]: # The Complete Project Gutenberg Works of Jane Austen (plain UTF-8)
JANE_AUSTEN_URL = "https://www.gutenberg.org/cache/epub/31100/pg31100.txt"
```

```
def get_gutenberg_text(url):
    """Download a Project Gutenberg plain-text file and return only the book
    text.

    Strip Gutenberg boilerplate using the standard START/END markers, normalizing
    the text.

    """
    time.sleep(2)
    book_string = requests.get(url, timeout=60).text
    start_pat = r'\*\*\* START OF (?:THIS|THE) PROJECT GUTENBERG EBOOK .+ \*'
    end_pat = r'\*\*\* END OF (?:THIS|THE) PROJECT GUTENBERG EBOOK'
    s = book_string.replace('\r', '')
    text1 = re.split(start_pat, s, maxsplit=1)[1]
    text = re.split(end_pat, text1, maxsplit=1)[0]
    return text

corpus = get_gutenberg_text(JANE_AUSTEN_URL)
print(f"Corpus length (characters): {len(corpus):,}")
```

Corpus length (characters): 4,354,160

Need a python reminder? 🤖 Ask ChatGPT! 🤖

## Let's test our code!

In [12]: corpus[1000:1200]

Out[12]: 'urned over the almost endless creations\nof the last century; and there, i  
f every other leaf were powerless, he\ncould read his own history with an i  
nterest which never failed. This\nwas the page at whi'

## Tokenizing a corpus

- **Tokenize** = turn the string into a **list of tokens**.
- Our token list will be made up of:
  - **words:** pride, and, prejudice
  - **punctuation:** ! ,
  - **paragraph breaks:** \x03
- Let's take a minute to play with [OpenAI's tokenizer](#)

Corpus

Tokenize

Train

Evaluate

```
In [6]: def tokenize(book_string):  
        """Split text into tokens (words, punctuation) with paragraph START/END  
  
        Paragraph boundaries use ASCII 2 (start) and ASCII 3 (end) as special tokens.  
        Punctuation except underscore is split into its own tokens.  
        """  
        book_string = re.sub(r'\n[\n]+\s*\n*', ' \x03 \x02', book_string).strip()
```

```

book_string = re.sub(r'\s+', ' ', book_string)
new_string = re.findall(r'([^\w ]{1}|[\w]+)', book_string)
if new_string[0] != '\x03':
    new_string = ['\x03', '\x02'] + new_string
if new_string[-1] != '\x02':
    new_string = new_string + ['\x03', '\x02']
return new_string[1:-1]

tokens = tokenize(corpus)
print(f"Token count: {len(tokens):,}")

```

Token count: 964,596

Need a python reminder? 🤖 Ask ChatGPT! 🤖

## Let's test our code!

In [7]: `print(tokens[1000:1020])`

```

['and', 'one', 'remained', 'a', 'widower', ',', 'the', 'other', 'a', 'wido
w', '.', '\x03', '\x02', 'That', 'Lady', 'Russell', ',', 'of', 'steady', 'ag
e']

```

## 🧠 Activity question 🧠

When babies need to learn language, do they need to "tokenize"? What would it look like?

Discuss in groups of 2-4 and send answers to the google form.



## Preview: Statistical learning

- Humans learn words (or "tokens") by extracting patterns from experience
  - Continuous speech: unbroken stream of sounds without explicit spaces or labels, and they split words.
  - Multiple cues: Real "human tokenization" blends statistics with everything else going on in a brain: vision, tone etc.

## Training

- Reminder: conditional probabilities and the chain rule.

- Joint probability = multiply  $P(\text{next token} \mid \text{earlier tokens})$  along a sequence.

```
>>> corpus = 'My name is Brendan. Her name is Jane. Their name is Kai.'
>>> tokens = tokenize(corpus)
>>> tokens
['\x02', 'My', 'name', 'is', 'Brendan', '.', 'Her', 'name', 'is', 'Jane', '.', 'Their', 'name', 'is', 'Kai', '.', '\x03']
```

Corpus

Tokenize

Train

Evaluate

## Chain rule on a phrase

$$\begin{aligned}P(\text{My name is Brendan}) &= P(\text{My}) \cdot P(\text{name} \mid \text{My}) \\&\quad \cdot P(\text{is} \mid \text{My name}) \cdot P(\text{Brendan} \mid \text{name is}) \\&\quad \cdot P(. \mid \text{is Brendan}) \\&= \frac{1}{17} \cdot 1 \cdot 1 \\&\quad \cdot \frac{1}{3} \cdot 1 \\&= \frac{1}{51}\end{aligned}$$

## Let's go back to Jane Austen!

```
In [8]: # N-gram order: 4 means we condition on up to 3 previous tokens.  
N = 4  
  
#write training data and a vocab dictionary  
train_data, vocab = padded_everygram_pipeline(N, [tokens])  
lm = MLE(N)  
lm.fit(train_data, vocab)
```

## Evaluation

- Let's start with some basic checks to see how our model did
- Later in the class we'll get to metrics for evaluation

Corpus

Tokenize

Train

Evaluate

```
In [9]: # Now let's check a phrase we know and fetch the most likely next word
prompt = ["I", "have", "no"]

vocab_tokens = [w for w in lm.vocab if w not in {"<s>", "</s>"}]
prediction = max(vocab_tokens, key=lambda w: lm.score(w, prompt))

print(
```

```

    {
        "prompt": prompt,
        "most_likely_next_token": prediction,
        "probability": round(lm.score(prediction, prompt), 3),
    }
)

```

```
{'prompt': ['I', 'have', 'no'], 'most_likely_next_token': 'doubt', 'probability': 0.255}
```

In [10]: *# Top-10 next-token candidates with probabilities for the same prompt.*

```

## generate a dictionary
vocab_tokens = [w for w in lm.vocab if w not in {"<s>", "</s>"}]

## create a list of tuples containing the words and their probability
scored = [(w, round(lm.score(w, prompt), 3)) for w in vocab_tokens]

## sort by most common
top10_next_tokens = sorted(scored, key=lambda x: x[1], reverse=True)[:10]

print("prompt:", prompt)
for token in top10_next_tokens:
    print(token)

```

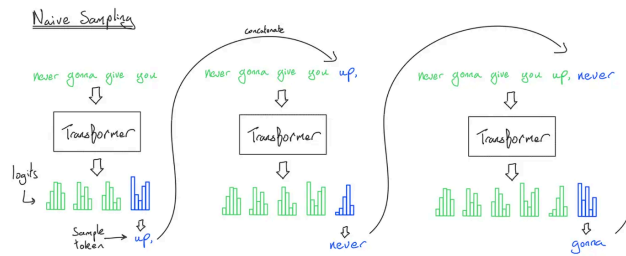
```

prompt: ['I', 'have', 'no']
('doubt', 0.255)
('idea', 0.082)
('reason', 0.064)
('right', 0.045)
('pleasure', 0.045)
('wish', 0.036)
('notion', 0.036)
('such', 0.027)
('hesitation', 0.027)
('patience', 0.027)

```

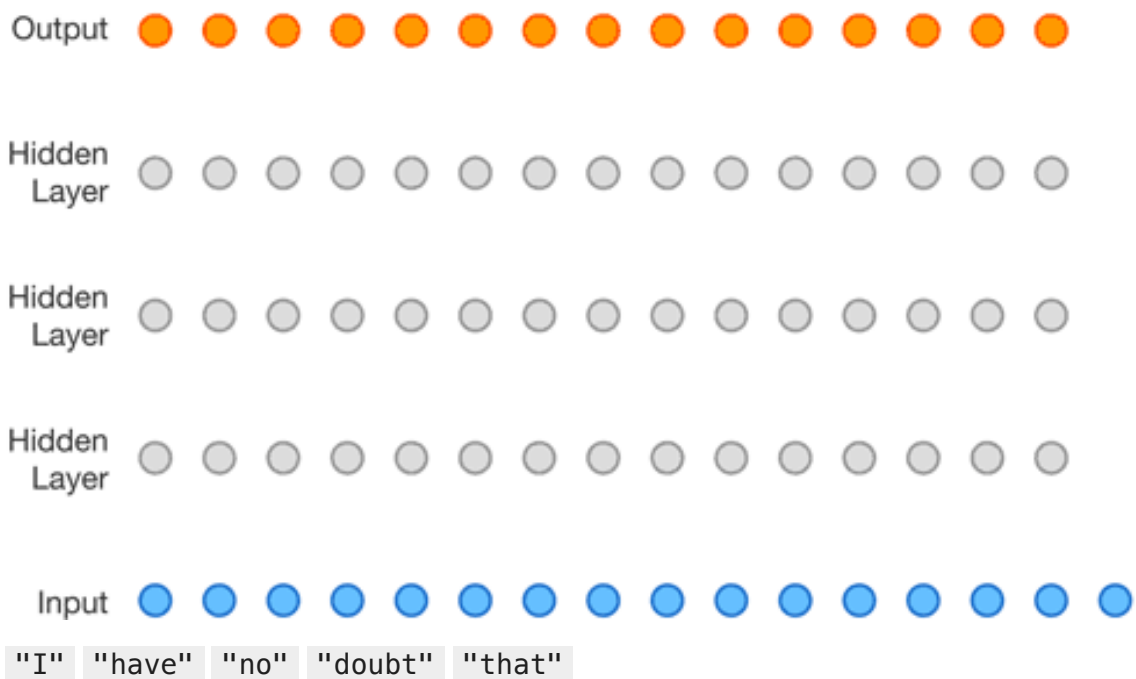
## From counts to guesses

The model is drawing from this as a **probability distribution**, so if the user says "I have no" then the language model will guess that you will say doubt 25.5% of the time.



## Self-supervised neural networks

- Intermediate layers between the input and output apply weighted sums and act like "neurons". We talk about this more in week 7!



## 🧠 Human Language 🧠

Is human language just...prediction? How do you think human language is similar or different to the predictive model we just built?

- Discuss in groups of 2-4 and send answers to the google form.



## A few possible answers:

- Similarities: Humans can predict the next word in the sentence and research shows that this is an important part of language development
- Differences: Humans have much more context than 3 words, abstract syntax (how we order words), and many other things we'll talk about in this class!

## We built a language model! How is it different than an LLM?

| Step     | N-gram           | LLMs                          |
|----------|------------------|-------------------------------|
| Corpus   | 1 million tokens | 10 trillion tokens            |
| Tokenize | words            | embeddings                    |
| Train    | chain-rule       | Deep neural net (Transformer) |
| Evaluate | Perplexity       | Perplexity                    |

## Preview: Self-attention & Syntax

- **Self-attention:** nodes **reweights** other positions with context
  - context windows
- **Syntax / pragmatics:** word order and sentence context affect how humans process language
  - working memory

## 🧠 Activity question 🧠

How do we think our Jane Austen dataset impacts our model outputs? What would happen if we used Shakespeare? Reddit? Instagram comments?

Take a minute to think and send answers to the google form or raise your hand.



## Data biases and selection

- "pleasure" is probably not the 5th most common word to follow "I have no"
- Data choices impact our model!

## In the news

- Recent article about Claude resorting to blackmail if told it will be deleted
- [BBC News — link](#)



# Hallucination example

- Biases in data can lead to **hallucinations** where LLMs provide you with incorrect information.

Lawyer used ChatGPT and ChatGPT made up some legal cases. ChatGPT was predicting words in a reasonable order. NYTimes link

## Other food for thought:

- Which languages does the LLM have access to?
- Of all the documents in the corpus, which communities are represented? Which authors? Whose facts?
- Does private data go into the model? What might be the consequences?

## 🧠 Activity question 🧠

Do you think there is bias in human language learning? What would that look like?

Discuss in groups of 2-4 and send answers to the chat.



## Preview: Sociolinguistics

- **Sounds / phonetics:** Your **accent** is driven by your input data.
- **Words and grammar:** Kids learn the **vocabulary** their community uses
  - You might say "cheugy" but I honestly don't know how to use it because I don't get any input in that way
- **Beliefs (politics, religion):** Trusted voices teach **what's true or sacred**; language carries **ideology** and **in-group norms**.

## Summary

- Pre-training is where a language model learns to **predict** language!
- N grams are a useful precursor to LLMs.
- Human language learning and LLM language learning have lots of parallels.

## What's coming up in the class?

- **Fine-tuning:** turning a language model into a conversational bot or otherwise shaping it to a purpose
  - how do we get from language prediction to a helpful chatbot?
- **Model evaluation:** How do we evaluate whether an LLM model is "good", human like
  - surprisal & perplexity
  - cognitive metrics

## Other links

- OpenAI on [why language models hallucinate](#)